

PMARS

Phase 2 Software Simulation Plan

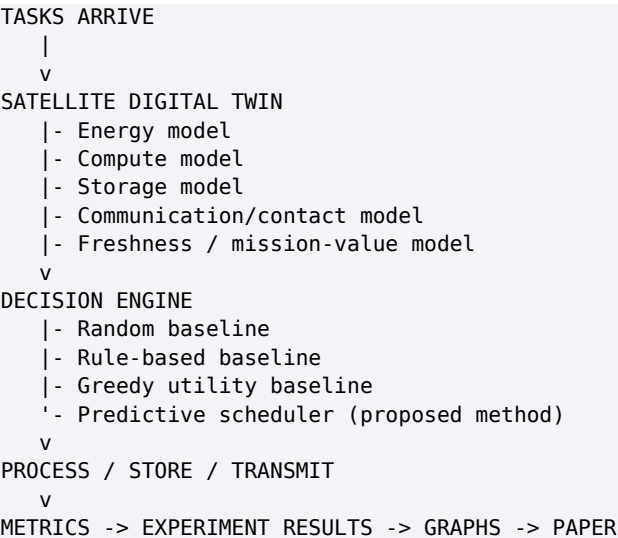
IASTAM 6.0 - Track 1, Problem 1: Process or Transmit?

Phase 2 objective: build a 100% software digital twin first, generate defensible preliminary results for the interim paper, then validate selected models on Raspberry Pi 4 in Phase 3.

Prepared for the project team - 15 September 2026

1. What You Are Building Now

For Phase 2, do not wait for the Raspberry Pi. Build a discrete-event software simulation that represents a resource-constrained satellite and evaluates scheduling strategies for PROCESS, STORE, and TRANSMIT. The simulation should produce repeatable experiments and quantitative early results for the interim research paper.



The challenge specification supports this approach: Phase 2 asks teams to set up a simulation or experimental environment, begin implementation, and provide preliminary results. For Problem 1, official evaluation indicators include decision quality, completed-task rate, estimated energy consumption, latency, and resource utilization.

2. Recommended Software Stack

Component	Recommended choice	Purpose
Simulation engine	Python + SimPy	Discrete-event simulation; fast execution of long virtual missions.
Numerical work	NumPy + pandas	State updates, datasets, metrics, aggregation.
Optimization	OR-Tools / custom look-ahead search	Greedy and predictive scheduling.
ML (optional in Phase 2)	scikit-learn / XGBoost	Predict PROCESS or TRANSMIT cost from simulated or later measured data.
Plots	Matplotlib	Paper-ready result figures.
Config	YAML / JSON	Repeatable scenarios and scheduler parameters.
Testing	pytest	Unit and integration tests.

3. Satellite Digital Twin

Create a Satellite object whose state changes over simulated time. The scheduler must never read arbitrary global variables; it receives a clean state snapshot.

```
SatelliteState = {
    time_s,
    energy_j, battery_percent, solar_power_w,
    cpu_available, memory_available_mb, temperature_c,
    storage_free_mb, queued_storage_mb,
```

```

contact_available, bandwidth_mbps, latency_ms,
contact_remaining_s, next_contact_s,
queue_length, critical_task_count
}

```

4. Common Task Model

Every workload should use the same task contract. This is critical for fair algorithm comparison.

```

Task = {
  id,
  type,
  input_size_mb,
  processing_demand,
  base_processing_time_s,
  processing_energy_estimate_j,
  processed_output_mb,
  priority,
  mission_value,
  deadline_s,
  created_at_s,
  freshness_decay
}

```

Task class	Typical characteristics	Why it matters
Telemetry packet	Tiny data, cheap processing, medium priority	Tests lightweight tasks and queue overhead.
Earth-observation image	Large raw data, expensive processing, high reduction after processing	Creates PROCESS vs TRANSMIT trade-off.
Emergency detection	Medium data, critical priority, short deadline	Tests urgency and mission value.
Scientific image	Very large data, lower urgency, long deadline	Makes STORE a potentially rational action.

5. Task Generator

- Generate tasks automatically; do not rely on a user clicking “Create Task”.
- Use randomized inter-arrival times or a Poisson process.
- Randomize task type, size, priority, deadline, mission value, and processing demand.
- Use fixed random seeds so each scheduler receives identical tasks.

```

Example:
seed = 42
mission_duration = 3600 simulated seconds
new task every 5-25 simulated seconds
critical-task probability = 0.10-0.20

```

6. Environment Models

6.1 Energy Model

For Phase 2, use explicit simulated assumptions and label them as assumptions in the paper. Do not present them as measured Raspberry Pi values.

```

Initial energy reservoir = configurable
Idle power = configurable
PROCESS power = configurable

```

TRANSMIT power = configurable
 Energy(action) = Power(action) x Duration(action)

- Add sunlight and eclipse periods.
- During sunlight, add charging power; during eclipse, solar generation is zero.
- Keep battery capacity bounded between 0 and its maximum.

6.2 Communication / Contact Model

Communication must be intermittent and time-varying. A contact window should have a start, end, and bandwidth profile.

0-100 s: NO CONTACT
 100-220 s: CONTACT, bandwidth rises from 1 to 10 Mbps then falls
 220-450 s: NO CONTACT
 450-610 s: CONTACT, different profile

Transmission time can be approximated initially as: $T_{tx} = (\text{DataSize_MB} \times 8) / \text{Bandwidth_Mbps} + \text{latency}$

If predicted transfer time exceeds contact remaining, the scheduler should treat completion as unlikely or impossible.

6.3 Compute Model

PROCESS should depend on task demand and currently available compute. $T_{process} = \text{BaseProcessingTime} / \text{AvailableComputeFraction}$

- PROCESS consumes energy and time.
- PROCESS may greatly reduce output size, which can make a later TRANSMIT much cheaper.

6.4 Storage and Freshness Model

STORE is not free. It consumes storage capacity and delays information delivery. Introduce mission-value decay so waiting can reduce usefulness. $V(t) = V_0 \times \exp(-\lambda \times \text{age})$

- Critical events: high freshness decay.
- Routine scientific data: low freshness decay.
- Stored tasks remain pending and are re-evaluated later.

6.5 Optional Thermal Model

For a stronger simulation, include a simple temperature state that increases during heavy processing and cools while idle. This lets the proposed scheduler delay a heavy task even when compute is technically available.

7. Define the Three Actions

Action	What the simulator does	State effects
PROCESS	Consume compute for predicted duration; reduce energy; generate smaller processed output.	Time advances; energy falls; result may enter a waiting-downlink queue.
STORE	Keep original or processed data onboard and postpone decision.	Storage decreases; age increases; mission value may decay.
TRANSMIT	Attempt to send raw/processed payload through current contact.	Time and energy consumed; may fail if contact ends; successful task leaves onboard queue.

8. Scheduler Algorithms

8.1 Random Scheduler - Sanity Baseline

Randomly selects PROCESS, STORE, or TRANSMIT subject to basic feasibility. It is not a serious competitor; it provides a weak reference and catches simulator bugs.

8.2 Rule-Based Scheduler - Engineering Baseline

```
Example logic:
IF no contact:
    PROCESS if enough energy and deadline is near
    otherwise STORE
ELSE IF transmission can finish before contact end:
    TRANSMIT if processing is expensive or deadline is urgent
ELSE:
    PROCESS or STORE
```

Keep this baseline reasonable. Do not intentionally make it weak.

8.3 Greedy Utility Scheduler - Strong Baseline

Evaluate only the immediate action and choose the largest normalized utility. $U(a) = w_v \cdot \text{Value} - w_e \cdot \text{Energy} - w_l \cdot \text{Latency} - w_b \cdot \text{Bandwidth} - w_f \cdot \text{FreshnessLoss} - w_d \cdot \text{DeadlineRisk}$

Normalize terms so one unit does not dominate only because of scale. Document all weights and run sensitivity checks.

8.4 Predictive Scheduler - Proposed Phase 2 Method

This is the main early contribution. It should consider future energy and contact conditions rather than only the current state.

```
Prediction horizon example: 300 s
Candidate plans:
    PROCESS now -> TRANSMIT later
    STORE -> PROCESS after sunlight -> TRANSMIT
    TRANSMIT now
    STORE until next contact
```

```
For each candidate plan:
    predict energy, latency, freshness loss, deadline risk, and mission value
Choose the plan with highest expected utility
Execute only the first action
Re-plan when state changes
```

For Phase 2, implement this with short look-ahead search. Reinforcement learning is optional later; do not block early results on RL.

9. Experiment Scenarios

Scenario	Configuration	Question answered
S1 - Normal	Moderate workload, moderate energy, regular contacts	Does the scheduler behave sensibly in nominal conditions?
S2 - Low Energy	Long eclipse / weak charging, normal workload	Can it preserve high-value tasks under energy scarcity?
S3 - Poor Communication	Few contacts, low bandwidth, high latency	Does onboard processing reduce dependence on downlink?

S4 - Task Burst	Large sudden increase in arrivals	Does the queue remain stable and prioritize correctly?
S5 - Critical Mission	Many high-value, short-deadline tasks	Does the scheduler protect critical mission value?

10. Fair Comparison Protocol

1. Use the exact same generated tasks, initial energy, contact windows, and random seed for all schedulers.
2. Run each scheduler under every scenario.
3. Repeat each scheduler/scenario combination across multiple random seeds (for example, 10 seeds).
4. Report mean, standard deviation, and where useful confidence intervals.
5. Never select only the run that makes the proposed method look best.

11. Metrics to Calculate Automatically

Metric	Definition / reason
Task completion rate	Percentage of tasks completed successfully.
Critical-task completion rate	Completion specifically for highest-priority tasks.
Deadline miss rate	How often useful tasks become late.
Average / P95 latency	Typical and worst-case completion delay.
Total energy consumed	Core official resource metric.
Energy per completed task	Efficiency indicator.
Total data transmitted	Measures communication burden.
Peak storage utilization	Shows whether STORE decisions cause backlog.
Average compute utilization	Official resource-utilization perspective.
Average freshness / information age	Measures damage caused by waiting.
Total mission value completed	Primary mission-level effectiveness measure.
Mission Efficiency	Derived metric: completed mission value / energy consumed.

12. Early Results Required for the Interim Paper

Before writing the paper, the software team should produce at least the following evidence:

Result	Minimum acceptable Phase 2 output
Simulator validity	Tasks arrive, resources evolve, communication opens/closes, and all three actions change state correctly.
Baseline comparison	Random, Rule-Based, and Greedy run successfully on the same scenario.
Proposed scheduler	Predictive scheduler runs end-to-end on at least the five scenarios.
Repeated experiments	Multiple seeds per scenario; results saved automatically.
Preliminary figures	At minimum: mission value, task completion, energy,

	latency/deadline misses.
Interpretation	Explain where predictive scheduling helps and where it does not.

13. Recommended Paper Figures

- Figure 1: Phase 2 digital-twin architecture.
- Figure 2: Example time-varying contact/bandwidth and sunlight/eclipses.
- Figure 3: Total mission value completed by scheduler.
- Figure 4: Task completion and critical-task completion.
- Figure 5: Energy consumed or Mission Efficiency.
- Figure 6: Deadline miss rate / latency.
- Figure 7: One illustrative task where Predictive chooses STORE -> PROCESS -> TRANSMIT and Greedy makes a poorer immediate choice.

14. Software Repository Structure

```

pmars-simulation/
|-- core/
|   |-- satellite.py
|   |-- task.py
|   |-- task_queue.py
|   '-- simulation.py
|-- environment/
|   |-- energy.py
|   |-- communication.py
|   |-- storage.py
|   |-- compute.py
|   '-- thermal.py
|-- schedulers/
|   |-- random_scheduler.py
|   |-- rule_scheduler.py
|   |-- greedy_scheduler.py
|   '-- predictive_scheduler.py
|-- scenarios/
|   |-- normal.yaml
|   |-- low_energy.yaml
|   |-- poor_link.yaml
|   |-- task_burst.yaml
|   '-- critical.yaml
|-- experiments/
|   |-- runner.py
|   |-- metrics.py
|   '-- results/
|-- analysis/
|   |-- statistics.py
|   '-- plots.py
|-- tests/
'-- README.md

```

15. Implementation Order - Do This First

Priority	Software work	Definition of done
1	Task model + satellite state + deterministic simulation clock	A single task can enter the queue and the state updates over time.
2	Energy + contact/bandwidth + storage	One hour of virtual mission runs without

	models	manual input.
3	PROCESS / STORE / TRANSMIT state transitions	Each action produces correct time, energy, storage, and completion effects.
4	Rule-based + Greedy baselines	Same scenario can be replayed under both schedulers.
5	Predictive look-ahead scheduler	Can choose a future-aware plan and re-plan.
6	Experiment runner + metrics	One command runs scheduler/scenario/seed and writes results.
7	Batch experiments	All scenarios x all schedulers x multiple seeds run automatically.
8	Plots + statistics	Figures and summary tables are generated from saved results.

16. Phase 2 Timeline (Software Only)

Target date	Required software milestone
16 Sep	Architecture, state/task contracts, repository, scenario format frozen.
17 Sep	Digital twin + task generator + energy/storage/contact models running.
18 Sep	PROCESS / STORE / TRANSMIT transitions verified; Rule scheduler complete.
19 Sep	Greedy utility scheduler + experiment runner + automatic metrics.
20-21 Sep	Predictive look-ahead scheduler implemented and debugged.
22 Sep	First full algorithm comparison across all scenarios.
23 Sep	Repeated runs, statistics, bug fixes, preliminary figures.
24 Sep	Software freeze for Phase 2 results.
24-25 Sep	Paper figures, result interpretation, pitch visuals.
26 Sep	Final verification and submission.

17. What You Can Honestly Claim in the Interim Paper

Recommended wording:

"A discrete-event simulation environment was developed to evaluate scheduling strategies before deployment on the physical onboard-computing demonstrator. The environment models task arrivals, onboard computing capacity, energy availability, storage limitations, dynamic communication windows, bandwidth variations, deadlines, and information freshness. Preliminary experiments compare rule-based, greedy, and predictive scheduling strategies under identical mission scenarios."

Then state clearly that energy and performance coefficients used in Phase 2 are simulation assumptions or literature-informed parameters, and that Phase 3 will replace selected parameters with Raspberry Pi 4 measurements.

18. What Not to Waste Time on Yet

- Do not build a desktop app for Raspberry Pi.
- Do not spend the first week on a beautiful dashboard.
- Do not start with reinforcement learning before baselines and the simulator work.
- Do not invent “real measured” energy values before hardware profiling.
- Do not run each scheduler on different random tasks.
- Do not report one lucky simulation run as scientific evidence.
- Do not hard-code the expected best decision into scenarios.

19. Phase 2 Software Completion Checklist

- ☐ Discrete-event simulator runs unattended.
- ☐ Task model is shared and documented.
- ☐ Energy, compute, storage, and dynamic communication models work.
- ☐ PROCESS, STORE, and TRANSMIT update the simulated mission correctly.
- ☐ Random, Rule-Based, and Greedy baselines work.
- ☐ Predictive scheduler runs and considers future conditions.
- ☐ At least five scenarios are implemented.
- ☐ Experiments are reproducible with seeds/config files.
- ☐ Batch runner executes repeated experiments automatically.
- ☐ Metrics are calculated automatically.
- ☐ Preliminary graphs are generated from result files.
- ☐ Paper clearly distinguishes simulation assumptions from hardware measurements.
- ☐ Phase 3 plan explains Raspberry Pi 4 validation.

20. Source Basis

Challenge requirements in this plan are grounded in the uploaded IASTAM 6.0 Technical Challenge Specification Book: Track 1 / Problem 1 defines PROCESS, STORE, and TRANSMIT decisions and lists resource constraints and evaluation indicators; Phase 2 requires state-of-the-art research, a simulation or experimental environment, initial development, and preliminary results; the general scoring grid rewards technical architecture, feasibility, prototype progress, methodology, and presentation.

Reference: IASTAM 6.0 Technical Challenge Specification Book, 2026, especially pages 6, 12-14.